Relatório de firmware — Terminal de Acesso (ESP32)

Nov 6, 2026 · @Pedro Henrique

Este relatório descreve, passo a passo, como construir o firmware do Terminal de Acesso a partir da montagem na protoboard: ESP32 DevKit V1 com RC522, display OLED, GPS NEO-6M, servo SG90 e buzzer, sem LEDs.

Resumo

O firmware transforma o ESP32 no Terminal de Acesso: mostra o QR Code que o backend gera, lê tags NFC, informa a posição GPS e aciona a catraca (servo) com sinal sonoro (buzzer) e visual (display). O terminal nunca decide sozinho se libera alguém: toda decisão vem do backend, e sem rede o acesso é negado.

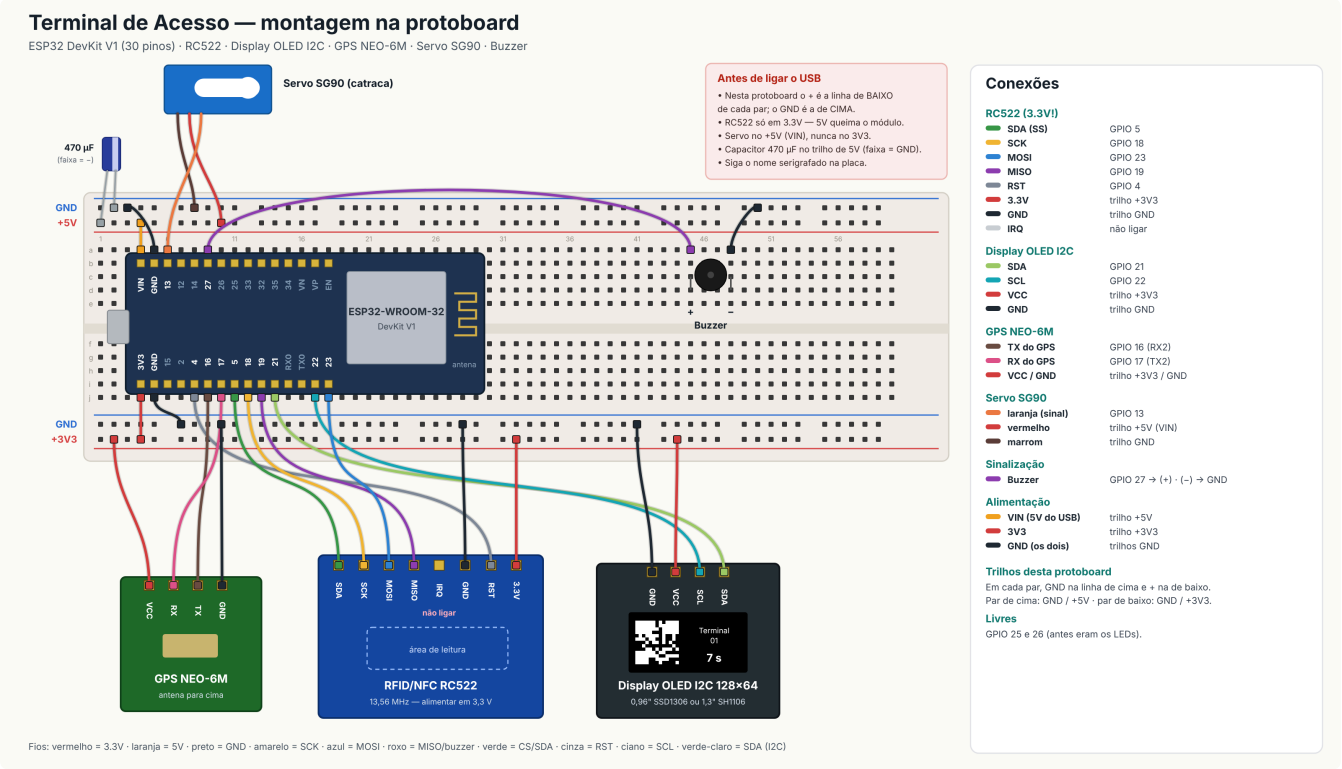
A comunicação é por HTTP(S) com polling: o terminal pergunta ao backend de tempos em tempos se há QR novo ou decisão pendente. O código de referência é o `terminal.ino` já entregue; este relatório explica cada parte dele e a ordem para construí-lo e testá-lo.

Requisito	O que o firmware faz	Sprint
RF11	Reúne os 7 itens mínimos do terminal: processamento, rede, tela, NFC, sinalização, GPS e abertura	1 a 3
RF12	Pede um QR novo a cada 10 s e desenha no OLED; nunca mostra QR vencido	2
RF13 / RF15 / RF16	Consulta a decisão do QR lido pelo app; abre a catraca ou sinaliza a negação	2
RF10 / RF14	Lê o UID da tag no RC522 (com debounce) e pede a validação ao backend	3
RNF02	Credenciais fora do código, chave própria do terminal, HTTPS	2 a 3
RNF03	Reconecta sozinho, nega quando está sem rede; fila local de eventos	3
RNF07	Loop sem espera ativa, brilho moderado, intervalos ajustáveis	3

A Sprint Review 2 (11/11) precisa do fluxo do QR ponta a ponta. NFC, fila offline e ajustes de energia ficam para a Sprint Review 3 (25/11).

Hardware de referência

O firmware foi escrito para exatamente esta montagem. Se algum fio mudar de pino, mude também a constante correspondente no topo do `terminal.ino`.



Pinagem usada pelo firmware

Constante no código	GPIO	Módulo e pino	Interface	Observação
`PIN_RC522_SS`	5	RC522 SDA (SS)	SPI	Pino de boot do ESP32; funciona bem com o RC522
`PIN_SPI_SCK`	18	RC522 SCK	SPI	
`PIN_SPI_MOSI`	23	RC522 MOSI	SPI	
`PIN_SPI_MISO`	19	RC522 MISO	SPI	
`PIN_RC522_RST`	4	RC522 RST	Saída digital	O IRQ do RC522 fica desligado

Constante no código	GPIO	Módulo e pino	Interface	Observação
`PIN_OLED_SDA`	21	OLED SDA	I2C	Endereço normal do OLED: 0x3C
`PIN_OLED_SCL`	22	OLED SCL	I2C	
`PIN_GPS_RX`	16 (RX2)	TX do GPS	UART2, 9600 baud	O TX de um vai no RX do outro
`PIN_GPS_TX`	17 (TX2)	RX do GPS	UART2	
`PIN_SERVO`	13	Servo, fio laranja	PWM 50 Hz	
`PIN_BUZZER`	27	Buzzer (+)	Saída digital ou PWM	PWM só se o buzzer for passivo
—	25 e 26	Livres	—	Eram dos LEDs; podem voltar a ser usados

Regras elétricas que o firmware assume

- **Trilhos desta protoboard:** em cada par, o GND é a linha de cima e o + é a linha de baixo. O par de cima é GND / +5V e o de baixo é GND / +3V3.
- **3,3 V:** RC522, OLED e GPS ficam no trilho +3V3. O RC522 queima com 5 V.
- **5 V:** só o servo usa o trilho +5V, que vem do pino VIN (5 V da USB).
- **Capacitor de 470 µF** no trilho de 5 V, com a faixa (−) no GND. Ele segura a tensão quando o servo arranca; sem ele o ESP32 pode reiniciar (erro "Brownout detector was triggered").
- **GND comum:** todos os GND (ESP32, módulos, servo, buzzer, capacitor) ligados entre si.
- **Alimentação:** cabo USB de dados direto no computador ou num power bank. Uma porta USB 2.0 de computador entrega cerca de 500 mA, então a catraca não deve ser forçada mecanicamente.
- **Pinos a evitar** se precisar remanejar algo: 6 a 11 (memória flash), 34 a 39 (só entrada) e 0, 2, 12 e 15 (definem o modo de boot).

Ambiente de desenvolvimento

O projeto usa a Arduino IDE 2 com o pacote de placas da Espressif. PlatformIO também funciona, com as mesmas bibliotecas.

Placa

1. Em *Arquivo → Preferências → URLs adicionais*, cole `https://espressif.github.io/arduino-esp32/package_esp32_index.json`.
2. Em *Ferramentas → Placa → Gerenciador de placas*, instale **esp32** (Espressif Systems).
3. Selecione a placa **DOIT ESP32 DEVKIT V1** (ou **ESP32 Dev Module**) e a porta COM que aparece ao ligar o cabo.
4. Se a porta não aparecer, instale o driver do chip USB da placa: CP2102 (Silicon Labs) ou CH340, conforme o que estiver escrito no chip ao lado do conector.
5. Monitor serial em **115200 baud**.

Bibliotecas

Instale pelo *Gerenciador de bibliotecas* (*Ferramentas → Gerenciar bibliotecas*).

Biblioteca	Autor	Include	Usada para	Cuidado
MFRC522	GithubCommunity (miguelbalboa)	`<MFRC522.h>`	Leitor NFC RC522	Não confundir com a MFRC522v2, que tem outra API
U8g2	oliver (olikraus)	`<U8g2lib.h>`	Display OLED	Um construtor para SSD1306 e outro para SH1106
TinyGPSPlus	Mikal Hart	`<TinyGPSPlus.h>`	GPS NEO-6M	
ESP32Servo	Kevin Harrington	`<ESP32Servo.h>`	Servo SG90	Com o pacote esp32 3.x, use a ESP32Servo 3.x
ArduinoJson	Benoit Blanchon	`<ArduinoJson.h>`	JSON da API	O código usa a versão 7 (`JsonDocument`); a 6 não compila

Biblioteca	Autor	Include	Usada para	Cuidado
QRCode	Richard Moore (ricmoo)	`"qrcode.h"`	Gerar o QR	Se der conflito de nome, veja Solução de problemas

WiFi, HTTPClient, WiFiClientSecure e SPI já vêm com o pacote esp32.

Pasta do projeto

A Arduino IDE exige que a pasta tenha o mesmo nome do `.ino`.

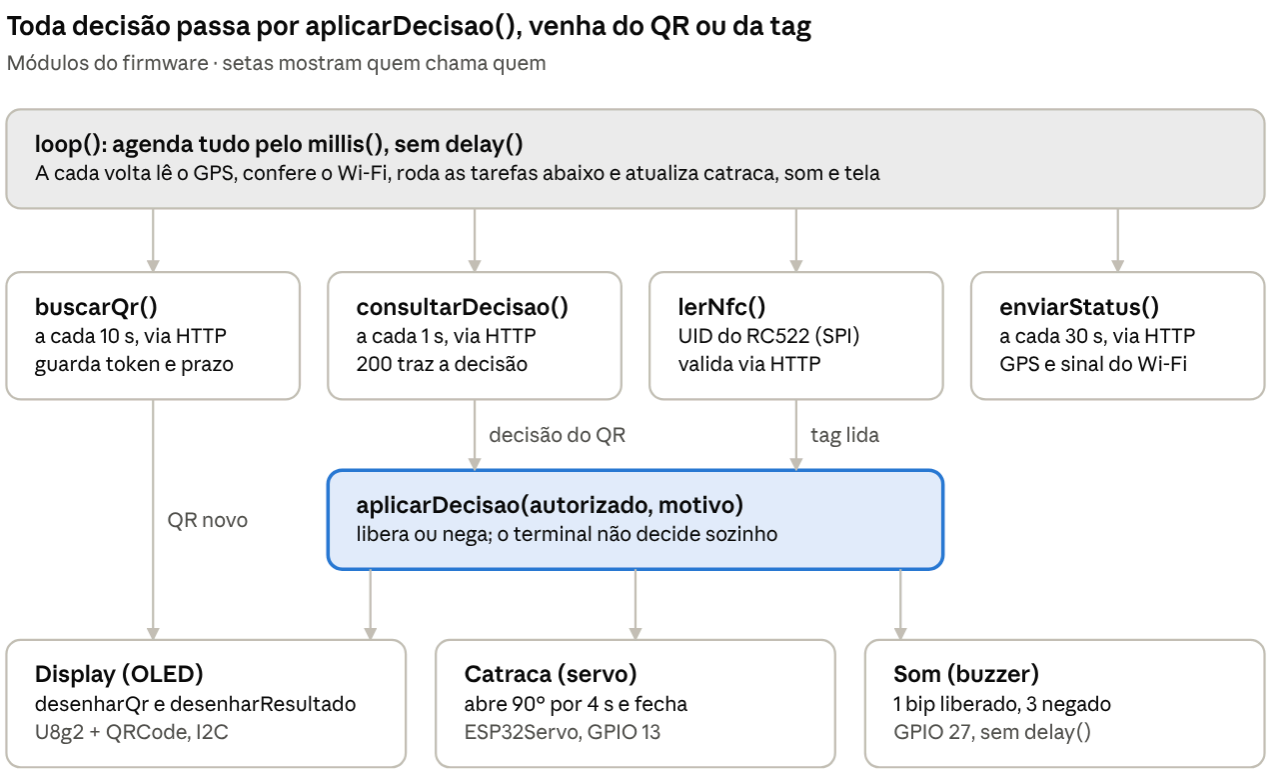
```
terminal/
├─ terminal.ino          # firmware
├─ secrets.example.h     # modelo de credenciais (vai para o Git)
├─ secrets.h             # credenciais reais (NÃO vai para o Git)
├─ .gitignore            # contém secrets.h
└─ README.md
```

Credenciais (`secrets.h`)

Copie `secrets.example.h` para `secrets.h` e preencha:

Campo	Exemplo	Observação
`WIFI_SSID` / `WIFI_SENHA`	`"RedeDoTime"`	O ESP32 só conecta em 2,4 GHz e não entra em redes com página de login. Para a demo, o roteador do celular é o mais seguro
`API_BASE_URL`	`"http://192.168.0.10:3000"`	Use o IP do computador na rede, nunca `localhost` (para o ESP32, `localhost` é ele mesmo). Sem barra no final
`TERMINAL_ID`	`"01"`	O mesmo ID cadastrado no backend (RF11)
`TERMINAL_API_KEY`	chave gerada no backend	Vai no cabeçalho `X-Terminal-Key` de toda requisição
`ROOT_CA` (opcional)	certificado raiz do servidor	Com HTTPS em produção, faz o ESP32 verificar que fala com o backend certo

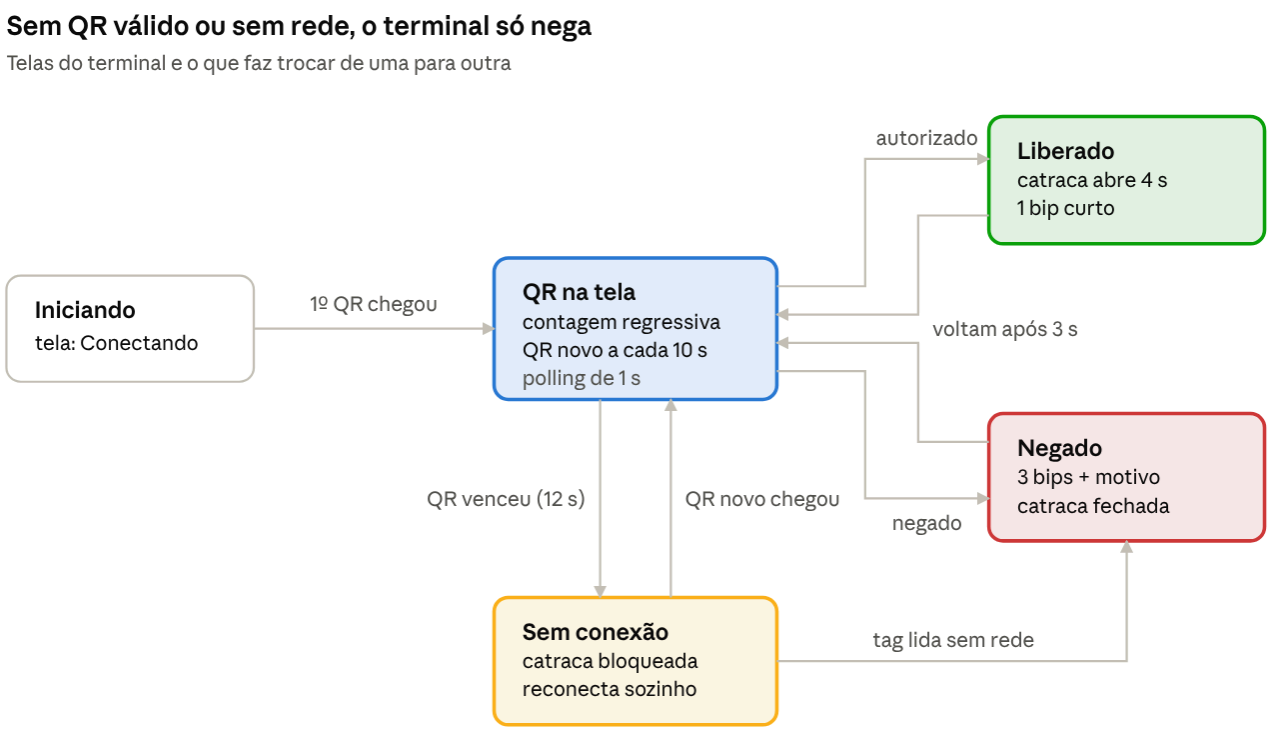
Arquitetura do firmware



arquitetura do firmware · módulos e chamadas

As quatro tarefas periódicas usam a mesma função `requisicao()`, que monta a chamada HTTP(S) com o cabeçalho `X-Terminal-Key` e timeout de 3 s. O QR vai direto para o display; decisões do QR e da tag passam por `aplicarDecisao()`, a única função que mexe na catraca.

Máquina de estados do terminal



máquina de estados · 5 telas

A variável `telaAtual` guarda o estado (`TELA_INICIANDO`, `TELA_QR`, `TELA_LIBERADO`, `TELA_NEGADO`, `TELA_SEM_CONEXAO`). Depois de Liberado ou Negado, o terminal volta para o QR se ele ainda for válido; se o QR venceu nesse meio-tempo, vai direto para Sem conexão.

Loop principal e temporização

O `loop()` nunca usa `delay()`. Cada tarefa guarda o instante da próxima execução e só roda quando ele chega. Assim o terminal continua lendo NFC e GPS enquanto o servo está aberto, o buzzer está tocando ou a tela está contando.

```
void loop() {
  while (Serial2.available()) gps.encode(Serial2.read());  // GPS: consome
tudo que chegou
  verificarWifi();                                         // detecta queda e
volta da rede

  if (passou(proximoQrEm))      { buscarQr();         proximoQrEm      =
millis() + INTERVALO_QR_MS; }
  if (passou(proximaDecisaoEm)) { consultarDecisao(); proximaDecisaoEm =
millis() + INTERVALO_DECISAO_MS; }
  if (passou(proximoStatusEm))  { enviarStatus();     proximoStatusEm  =
millis() + INTERVALO_STATUS_MS; }

  lerNfc();            // RC522
  atualizarCatraca();  // fecha o servo quando o tempo acaba
  atualizarSom();      // avança o padrão de bips
  atualizarTela();     // contagem do QR e volta da tela de resultado
}
```

A função `passou(t)` compara com `(long)(millis() - t) >= 0`. Essa forma continua certa quando o `millis()` estoura e volta a zero, depois de cerca de 49 dias ligado.

Intervalos

Constante	Valor	O que controla	Requisito
`INTERVALO_QR_MS`	10 000 ms	Pedido de QR novo	RF12
`TOLERANCIA_QR_MS`	2 000 ms	Folga antes de considerar o QR vencido e mostrar SEM CONEXÃO	RF12
`INTERVALO_DECISAO_MS`	1 000 ms	Polling da decisão do QR; é o atraso máximo para a catraca abrir	RF15
`INTERVALO_STATUS_MS`	30 000 ms	Envio de status e GPS	RF11
`TIMEOUT_HTTP_MS`	3 000 ms	Tempo máximo de espera de cada requisição	RNF03
`TEMPO_RESULTADO_MS`	3 000 ms	Tempo da tela LIBERADO/NEGADO	RF15 / RF16
`TEMPO_CATRACA_ABERTA_MS`	4 000 ms	Tempo com o servo aberto	RF15

Constante	Valor	O que controla	Requisito
`DEBOUNCE_NFC_MS`	3 000 ms	Janela em que a mesma tag é ignorada	RF10

O único ponto que bloqueia

Cada requisição HTTP trava o loop até a resposta chegar ou o timeout vencer. Com o backend fora do ar, cada consulta pode parar o terminal por até 3 s, e a leitura de NFC fica lenta nesse período.

- Para a Sprint 2 isso é aceitável: sem Wi-Fi o firmware nem tenta a requisição, e com o backend no ar a resposta leva poucas dezenas de milissegundos na rede local.
- Se incomodar, reduza `TIMEOUT_HTTP_MS` para 1 500 ms.
- Solução completa (Sprint 3): mover a comunicação para uma tarefa FreeRTOS no núcleo 0 do ESP32, deixando o loop só com periféricos.

Periféricos em detalhe

Cada periférico tem uma função de inicialização no `setup()` e uma função curta chamada pelo `loop()`.

Display OLED e QR Code (RF12)

- **Modelo:** deixe descomentada só a linha do seu display. SSD1306 é o de 0,96″; SH1106 é o de 1,3″. Com o construtor errado a imagem sai deslocada ou com uma faixa de lixo na lateral.

```
U8G2_SSD1306_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE,
PIN_OLED_SCL, PIN_OLED_SDA);  // 0,96"
// U8G2_SH1106_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE,
PIN_OLED_SCL, PIN_OLED_SDA); // 1,3"
```

- **Tamanho do QR:** o firmware usa a versão 3 do QR (29×29 módulos) com 2 pixels por módulo. Dá 58 px, mais 3 px de borda branca de cada lado: 64 px, a altura exata da tela. O texto do terminal e a contagem ficam à direita.
- **Limite do token:** a versão 3 com correção de erro baixa (`ECC_LOW`) comporta até 53 caracteres. Acima disso o firmware mostra "Token grande demais". Peça ao backend um token opaco curto, por exemplo 16 bytes aleatórios em base64url (22 caracteres), guardado no banco com validade e uso único.
- **Cores:** o QR é desenhado com módulos escuros sobre fundo aceso, que é o que as câmeras leem melhor.

- **Atualização da tela:** o display só é redesenhado quando o segundo da contagem muda ou quando a tela troca, para não ocupar o barramento I2C à toa.
- **QR vencido:** se não chegar QR novo até 10 s + 2 s de tolerância, a tela muda para SEM CONEXAO. O terminal nunca mostra um QR que o backend já não aceita.
- **Brilho:** `oled.setContrast(160)` deixa o brilho moderado (RNF07). Suba se a câmera do celular tiver dificuldade.

Leitor NFC RC522 (RF10 / RF14)

- **Inicialização:** `SPI.begin(18, 19, 23, 5)` e depois `rfid.PCD_Init()`. Em seguida o firmware lê o registrador de versão: `0x00` ou `0xFF` significa que o módulo não está respondendo (fio solto ou sem 3,3 V). Qualquer outro valor (`0x91`, `0x92` ou o de um clone, como `0x88` ou `0x12`) significa que a comunicação está ok.
- **Leitura:** a cada volta do loop, `PICC_IsNewCardPresent()` e `PICC_ReadCardSerial()`. O UID vira texto hexadecimal em maiúsculas, por exemplo `04A1B2C3`. Depois o firmware chama `PICC_HaltA()` e `PCD_StopCrypto1()` para liberar o cartão.
- **Debounce:** a mesma tag é ignorada por 3 s, para quem segura o cartão no leitor não gerar várias tentativas.
- **Tamanho do UID:** cartões MIFARE Classic têm 4 bytes (8 caracteres); tags NTAG têm 7 bytes (14 caracteres). O backend deve aceitar os dois.
- **Cadastro de tags:** o firmware não precisa de um modo especial. Uma tag desconhecida gera uma tentativa negada com o UID no evento; o administrador copia esse UID do histórico (ou do monitor serial) e vincula ao usuário no app.
- **Limitação conhecida:** o UID de muitas tags pode ser clonado. Para o escopo acadêmico isso fica documentado como risco aceito.

GPS NEO-6M (RF11)

- **Conexão:** `Serial2.begin(9600, SERIAL_8N1, 16, 17)`. O NEO-6M sai de fábrica a 9600 baud.
- **Leitura:** todo byte que chega vai para `gps.encode()`. A posição só é enviada se `gps.location.isValid()` e se tiver menos de 10 s.
- **Primeiro fix:** com céu aberto pode levar de 1 a alguns minutos depois de ligar. Dentro de prédio quase sempre não há fix; nesse caso o status vai com `gpsValido: false` e o backend usa as coordenadas do local cadastrado.
- **Sinal de vida:** na maioria das placas NEO-6M, o LED do módulo pisca uma vez por segundo quando há fix.
- **Fios trocados:** se `gps.charsProcessed()` continuar em zero depois de alguns segundos, o TX e o RX estão invertidos ou o módulo está sem alimentação.

Servo SG90 — catraca (RF15)

- **Configuração:** `catraca.setPeriodHertz(50)` e `catraca.attach(13, 500, 2400)`. Os valores 500 e 2400 µs são a faixa de pulso do SG90.
- **Ângulos:** `ANGULO_FECHADA = 0` e `ANGULO_ABERTA = 90`. Ajuste conforme a peça que representa a catraca.
- **Abertura:** `abrirCatraca()` gira para 90° e agenda o fechamento para 4 s depois; quem fecha é `atualizarCatraca()`, chamada no loop.
- **Melhoria de energia (opcional):** depois de fechar, `catraca.detach()` corta o sinal e o servo para de tremer e de consumir; antes de abrir, `attach()` de novo.

Buzzer (RF16)

- **Ativo ou passivo:** o ativo apita sozinho quando recebe tensão (costuma ter o fundo fechado e um adesivo). O passivo só faz um clique com tensão contínua e precisa de uma frequência (costuma mostrar a placa verde por baixo). Para passivo, mude `BUZZER_PASSIVO` para `true`; o firmware passa a usar `tone()`.
- **Padrões:** liberado = 1 bip de 150 ms. Negado = 3 bips de 400 ms com 150 ms de pausa. Os padrões são diferentes para quem não enxerga a tela (RNF04).
- **Sem travar:** um pequeno sequenciador (`tocar()` e `atualizarSom()`) liga e desliga o buzzer pelo `millis()`, sem `delay()`.
- **Corrente:** o GPIO do ESP32 fornece pouca corrente. Se o buzzer for de 5 V ou consumir mais de cerca de 20 mA, ligue-o por um transistor (por exemplo, 2N2222 com resistor de 1 kΩ na base).

Comunicação com o backend

O terminal usa quatro rotas, todas com prefixo `/api/v1` e com o cabeçalho `X-Terminal-Key`. Duas delas (`/decision` e `/status`) não estavam no guia original e precisam ser criadas no backend.

Rota	Quando	Envia	Resposta esperada	O que o firmware faz
`POST /terminals/:id/qr`	A cada 10 s	`{}`	`200 {"token":"...","expiraEmSeg":10}`	Desenha o QR novo. Qualquer outro código é ignorado e o QR antigo vence sozinho
`GET /terminals/:id/decision`	A cada 1 s	—	`204` sem corpo, ou `200 {"autorizado":true,"motivo":""}`	204: nada. 200: libera ou nega
`POST /access/nfc/validate`	Tag lida	`{"terminalId":"01","uid":"04A1B2C3"}`	`200` ou `403` com `{"autorizado":false,"motivo":"SEM_AUTORIZACAO"}`	Lê o JSON e aplica. Sem conexão: nega com "SEM CONEXAO"

Rota	Quando	Envia	Resposta esperada	O que o firmware faz
`POST /terminals/:id/status`	A cada 30 s	`{"gpsValido":true,"lat":-23.30,"lng":-45.96,"satelites":7,"rssi":-61,"versao":"0.1.0"}`	`200`	Só registra no monitor serial

Fluxo do QR, de ponta a ponta

1. O terminal pede um QR; o backend gera um token curto de uso único, salva com validade de 10 s e devolve.
2. O terminal desenha o QR e começa a contagem.
3. O usuário lê o QR no app; o app envia o token, a posição do celular e o JWT para `POST /access/qr/validate`.
4. O backend valida, registra o evento e guarda uma decisão pendente para aquele terminal.
5. Na próxima consulta a `/decision` (no máximo 1 s depois), o terminal recebe a decisão e abre a catraca ou sinaliza a negação.

Regras que o backend precisa seguir

- **Decisão entregue uma vez só:** ao responder `200` em `/decision`, o backend apaga a decisão pendente. Se não apagar, a catraca abre de novo a cada segundo.
- **Validade da decisão:** uma decisão que ninguém buscou em 5 s deve ser descartada, para o terminal não abrir sozinho minutos depois de voltar da queda de rede.
- **Corpo JSON também na negação:** em `/access/nfc/validate`, o `403` precisa trazer `autorizado` e `motivo`. Uma resposta sem JSON aparece como "ERRO".
- **Chave inválida:** responder `401`. O firmware não libera nada sem um `200` com `autorizado: true`.
- **Horário:** toda conta de validade é feita no backend. O terminal não usa relógio real.

Texto do motivo no display

O display mostra até 21 caracteres na linha do motivo; códigos maiores são cortados (`CREDENCIAL_REUTILIZADA` tem 22). Para mensagens mais claras, troque o código por um texto curto antes de desenhar:

```
String textoDoMotivo(const String& codigo) {
  if (codigo == "CREDENCIAL_INVALIDA")    return "QR invalido";
  if (codigo == "CREDENCIAL_EXPIRADA")    return "QR expirado";
  if (codigo == "CREDENCIAL_REUTILIZADA") return "QR ja usado";
  if (codigo == "SEM_AUTORIZACAO")        return "Sem autorizacao";
  if (codigo == "AUTORIZACAO_CANCELADA")  return "Autoriz. cancelada";
  if (codigo == "FORA_DO_PERIODO")        return "Fora do horario";
  if (codigo == "USOS_ESGOTADOS")         return "Usos esgotados";
  if (codigo == "DISTANTE_DO_TERMINAL")   return "Longe do terminal";
  if (codigo == "USUARIO_INATIVO")        return "Usuario inativo";
  return codigo;  // código desconhecido: mostra como veio
}
```

Chame `textoDoMotivo(motivo)` dentro de `desenharResultado()`. O monitor serial continua mostrando o código original.

Segurança, confiabilidade e energia

A regra geral do firmware é falhar fechado: qualquer resposta inesperada, JSON quebrado, timeout ou falta de rede mantém a catraca fechada.

Segurança (RNF02)

- **Credenciais fora do Git:** Wi-Fi, URL e chave ficam em `secrets.h`, listado no `.gitignore`. Só o `secrets.example.h` vai para o repositório.
- **Chave por terminal:** cada terminal tem a sua `TERMINAL_API_KEY`. O backend guarda apenas o hash dela; se vazar, basta gerar outra.
- **HTTPS:** em produção a URL começa com `https://`. Com `ROOT_CA` definido, o ESP32 verifica o certificado do servidor (`setCACert`). Sem ele, o firmware usa `setInsecure()`, que criptografa mas não verifica: aceitável só em desenvolvimento.
- **Nada sensível no monitor serial:** o firmware imprime UID e decisões, nunca a chave ou a senha do Wi-Fi.
- **Entrada validada:** token vazio ou maior que 53 caracteres não vira QR; JSON que não abre não libera nada.

Confiabilidade (RNF03)

- **Reconexão:** `WiFi.setAutoReconnect(true)` refaz a conexão sozinho. Quando a rede volta, `verificarWifi()` pede um QR novo na hora.
- **Sem rede:** o QR vence e a tela mostra SEM CONEXAO; tags lidas são negadas com sinal sonoro.

- **Fila local (Sprint 3):** toda tentativa precisa ser registrada (RF17), inclusive as negadas por falta de rede. Proposta:
  - guardar cada tentativa offline (UID, momento em `millis()`, motivo) em memória não volátil com a biblioteca `Preferences` ou em um arquivo no LittleFS, até 50 itens;
  - dar a cada item um identificador único (ID do terminal + contador salvo), para o backend descartar duplicados;
  - ao reconectar, enviar em lote para uma rota nova, por exemplo `POST /terminals/:id/events`, e apagar só o que o backend confirmar.
- **Travamento:** `enableLoopWDT()` no `setup()` reinicia o ESP32 se o loop ficar preso por vários segundos.

Energia (RNF07)

O terminal precisa ficar sempre pronto (QR a cada 10 s e leitura de NFC), então não usa deep sleep. A economia vem de não fazer trabalho à toa:

- loop sem espera ativa e display redesenhado só quando algo muda;
- brilho do OLED moderado (`setContrast(160)`);
- Wi-Fi no modo padrão de economia do modem, que desliga o rádio entre transmissões;
- status e GPS a cada 30 s, e não a cada volta do loop;
- servo desligado (`detach()`) quando fechado, se a peça se mantiver parada;
- melhoria seguinte: trocar o polling de 1 s por long polling ou MQTT, que corta a maior parte das requisições vazias.

Esses pontos servem de justificativa na documentação do RNF07. Se der tempo, meçam a corrente com um medidor USB nos dois modos (polling de 1 s e de 5 s) e registrem a diferença.

Roteiro de implementação

Monte e teste uma parte por vez, na ordem abaixo. Quando algo falhar, você sabe que o problema está na última peça adicionada. As etapas 1 a 8 fecham a Sprint Review 2 (11/11); as etapas 9 a 11 são da Sprint Review 3 (25/11).

1. **Placa e monitor serial.** Grave um sketch vazio que só imprime uma mensagem no `setup()`.
   - Pronto quando: a mensagem aparece no monitor serial a 115200.
2. **Display.** Inicialize o OLED e escreva um texto.

- Pronto quando: o texto aparece nítido. Se a tela ficar preta, rode um scanner I2C: o OLED deve aparecer em 0x3C.

3. **QR local.** Desenhe um QR com um texto fixo, por exemplo `TESTE-123`, usando a função `desenharQr()`.
   - Pronto quando: a câmera do celular lê `TESTE-123` na tela de 0,96".
4. **Servo e buzzer.** Chame `abrirCatraca()` e os dois padrões de som a partir de um comando no monitor serial.
   - Pronto quando: o servo vai a 90° e volta em 4 s, os bips são diferentes e o ESP32 não reinicia quando o servo mexe.
5. **Wi-Fi e backend no ar.** Conecte ao Wi-Fi e chame `GET /api/v1/health`.
   - Pronto quando: o monitor mostra o IP do ESP32 e o código 200. Este é o "terminal online" da demo.
6. **QR vindo do backend.** Ative `buscarQr()` com o servidor de teste abaixo ou com o backend real.
   - Pronto quando: o QR muda a cada 10 s; desligando o servidor, a tela mostra SEM CONEXAO em cerca de 12 s e volta sozinha ao religar.
7. **Decisão por polling.** Ative `consultarDecisao()`. Com o servidor de teste, abra `/teste/liberar` e `/teste/negar` no navegador do PC.
   - Pronto quando: a catraca abre em até 1 s, uma vez só por decisão, e a negação mostra o motivo com 3 bips.
8. **Ponta a ponta com o app.** Troque o servidor de teste pelo backend real e leia o QR pelo app.
   - Pronto quando: os cenários do Plano de testes passam. Esta é a demo da Sprint 2.
9. **NFC.** Ative `lerNfc()` primeiro só imprimindo o UID; depois ligue `validarNfc()`.
   - Pronto quando: cada tag mostra seu UID uma vez por aproximação, e a tag vinculada libera.
10. **GPS e status.** Ative `enviarStatus()` e deixe o GPS perto de uma janela.
    - Pronto quando: o backend recebe o status a cada 30 s, com `gpsValido: true` depois do fix.
11. **Robustez e energia.** Fila local de eventos, `enableLoopWDT()`, `detach()` do servo e medição de consumo.
    - Pronto quando: tentativas feitas sem rede aparecem no histórico depois que a rede volta, sem duplicar.

Servidor de teste para as etapas 6 e 7

Permite testar o firmware antes do backend real ficar pronto. Rode no PC com `npm i express` e `node mock-terminal.js`; libere a porta 3000 no firewall e use o IP do PC no `API_BASE_URL`.

```
// mock-terminal.js
const express = require('express');
const app = express();
app.use(express.json());

let pendente = null; // decisão esperando o terminal buscar

app.get('/api/v1/health', (req, res) => res.json({ ok: true }));

app.post('/api/v1/terminals/:id/qr', (req, res) => {
  res.json({ token: 'T-' + Date.now().toString(36), expiraEmSeg: 10 });
});

app.get('/api/v1/terminals/:id/decision', (req, res) => {
  if (!pendente) return res.status(204).end();
  const decisao = pendente;
  pendente = null; // entrega uma vez só
  res.json(decisao);
});

app.post('/api/v1/terminals/:id/status', (req, res) => {
  console.log('status', req.body);
  res.sendStatus(200);
});

app.post('/api/v1/access/nfc/validate', (req, res) => {
  console.log('NFC', req.body.uid);
  res.status(403).json({ autorizado: false, motivo: 'SEM_AUTORIZACAO' });
});

// Abra no navegador do PC para simular o app lendo o QR
app.get('/teste/liberar', (req, res) => { pendente = { autorizado: true,
motivo: '' }; res.send('liberar enviado'); });
app.get('/teste/negar', (req, res) => { pendente = { autorizado: false,
motivo: 'USOS_ESGOTADOS' }; res.send('negar enviado'); });

app.listen(3000, '0.0.0.0', () => console.log('Servidor de teste na porta
3000'));
```

Plano de testes

Testes manuais do terminal, para marcar antes de cada Sprint Review. Os testes automatizados exigidos pelo RP10 ficam no backend; aqui é a validação do hardware e do firmware.

Inicialização e hardware

- [ ] Ligado só na USB do computador, o ESP32 inicia sem reiniciar em loop
- [ ] O monitor serial mostra "RC522 ok" com uma versão diferente de 0x00 e 0xFF
- [ ] O OLED mostra texto nítido e o QR é lido pela câmera do celular

QR Code (RF12)

- [ ] O QR muda a cada 10 s e a contagem regressiva acompanha
- [ ] Com o backend desligado, a tela mostra SEM CONEXAO em até 12 s e nunca exibe QR vencido
- [ ] Religando o backend, o QR volta sem reiniciar o ESP32

Decisão do QR (RF13, RF15, RF16)

- [ ] Autorizado: a catraca abre em até 1 s, fecha após 4 s, toca 1 bip e mostra LIBERADO
- [ ] Negado: a catraca não se move, tocam 3 bips longos e a tela mostra NEGADO com o motivo
- [ ] Cada decisão abre a catraca uma vez só
- [ ] Dez aberturas seguidas sem o ESP32 reiniciar

NFC (RF10, RF14)

- [ ] Tag vinculada a uma autorização válida libera
- [ ] Tag desconhecida é negada e aparece no histórico com o UID
- [ ] Segurar a tag no leitor por 10 s gera uma tentativa só
- [ ] Sem rede, a tag é negada com sinal sonoro

GPS e status (RF11)

- [ ] O backend recebe o status a cada 30 s
- [ ] Ao ar livre, `gpsValido` vira `true` com coordenadas coerentes com o local

Rede e robustez (RNF03)

- [ ] Desligar o roteador por 1 minuto e religar: o terminal volta sozinho

- [ ] Com o backend fora, o terminal continua respondendo (tela atualiza, NFC nega)
- [ ] Uma hora ligado sem reiniciar (conferir no monitor serial)

Segurança (RNF02)

- [ ] `secrets.h` não aparece no `git status`
- [ ] Com a chave errada, o backend responde 401 e a catraca não abre

Solução de problemas

Os problemas mais comuns nesta montagem, agrupados por onde aparecem.

Sintoma	Causa provável	Correção
A porta COM não aparece	Cabo só de carga ou falta o driver do chip USB	Troque por um cabo de dados; instale o driver CP2102 ou CH340
Upload para em "Connecting..." ou dá "Wrong boot mode"	A placa não entrou no modo de gravação	Segure o botão BOOT quando aparecer "Connecting..." e solte quando começar a gravar; reduza a velocidade de upload para 115200
Erro de compilação com `JsonDocument`	ArduinoJson na versão 6	Atualize a ArduinoJson para a versão 7
Erro de compilação na ESP32Servo (por exemplo, `ledcSetup` não declarado)	ESP32Servo antiga com o pacote esp32 3.x	Atualize a ESP32Servo para a versão 3.x
Erro de compilação em `qrcode_initText` ou tipos em conflito	Outro arquivo `qrcode.h` com o mesmo nome entrou no lugar	Copie `qrcode.h` e `qrcode.c` da biblioteca para a pasta do projeto com outro nome (`qrcode_gen.h` e `qrcode_gen.c`), troque o `#include` dentro do `.c` e no `terminal.ino`
"Brownout detector was triggered" quando o servo mexe	Queda do 5 V no arranque do servo	Confira o capacitor de 470 µF e a polaridade; use outra porta USB ou o power bank

Sintoma	Causa provável	Correção
"Guru Meditation Error" e reinício	Erro no código (por exemplo, acesso fora de um vetor)	Copie o backtrace do monitor serial e decodifique com o ESP Exception Decoder
OLED totalmente preto	SDA e SCL trocados, sem 3,3 V ou endereço diferente	Confira 21 = SDA e 22 = SCL; rode um scanner I2C (esperado 0x3C)
OLED com faixa de lixo na lateral ou imagem deslocada	Display SH1106 usando o construtor do SSD1306	Troque para a linha do SH1106
O celular não lê o QR	Tela pequena, foco ou brilho	Aproxime até a câmera focar (5 a 10 cm); suba o `setContrast`; um display de 1,3" ajuda
A tela mostra "Token grande demais"	O backend manda um token com mais de 53 caracteres	Gere um token opaco curto no backend
A tela fica sempre em SEM CONEXAO	Wi-Fi de 5 GHz ou com página de login, senha errada, `localhost` na URL, firewall do PC, ou backend escutando só em 127.0.0.1	Use rede de 2,4 GHz, o IP do PC na URL, libere a porta no firewall e faça o backend escutar em 0.0.0.0. Veja o código HTTP no monitor serial: -1 é rede, 401 é chave, 404 é rota
"RC522 não respondeu"	Fio SPI solto, sem 3,3 V, RST desligado ou pinos do módulo sem solda	Confira 5, 18, 19, 23, 4 e o 3,3 V; solde os pinos do módulo se vieram soltos
O RC522 lê só às vezes	Tag longe ou metal perto da antena	Encoste a tag (alcance de cerca de 3 cm); afaste a antena de superfícies metálicas
GPS sem nenhum dado (`charsProcessed` em 0)	TX e RX invertidos ou módulo sem energia	O TX do GPS vai no GPIO 16; o RX do GPS vai no 17
GPS com dados mas sem posição	Sem visão do céu	Teste perto da janela ou ao ar livre e espere alguns minutos; dentro do prédio o backend usa as coordenadas do local

Sintoma	Causa provável	Correção
O servo treme parado	Ruído na alimentação ou sinal PWM sempre ativo	Use `detach()` com a catraca fechada; confira o GND comum
O buzzer só faz um clique	Buzzer passivo com `BUZZER_PASSIVO = false`	Mude para `true`
HTTPS retorna -1, mas HTTP funciona	Certificado em `ROOT_CA` errado ou vencido	Teste sem `ROOT_CA` (modo inseguro); se funcionar, pegue de novo o certificado raiz do servidor
A catraca abre sozinha várias vezes seguidas	O backend não apaga a decisão depois de entregar	Corrija a rota `/decision` para entregar cada decisão uma vez só

Fontes

As funções citadas foram conferidas no código-fonte de cada biblioteca:

- arduino-esp32 (Espressif): `tone()`, `noTone()` e `enableLoopWDT()`
- U8g2: construtores SSD1306 e SH1106 com pinos I2C
- MFRC522 (miguelbalboa/rfid): registrador de versão e valores de chips originais e clones
- TinyGPSPlus: `encode()`, `age()` e `charsProcessed()`
- ESP32Servo: `setPeriodHertz()`, `attach()` e `write()`
- ArduinoJson 7: classe `JsonDocument`
- QRCode (ricmoo): `qrcode_initText()`, `qrcode_getModule()` e `ECC_LOW`

Do próprio projeto: `terminal.ino`, `secrets.example.h` e `README.md` do firmware, e a imagem da montagem na protoboard.